# COMPSCI 546: Applied Information Retrieval - Spring 2026 (website)

## Assignment 5: Neural Information Retrieval (Total: 100 points)

**Description**

This assignment consists of programming and analytical questions on Neural Ranking Models. You will implement a bi-encoder dense retrieval model, build a FAISS index for efficient retrieval, and compare neural retrieval with traditional methods.

**Instructions**

- To start working on the assignment, you would first need to save the notebook to your local Google Drive. For this purpose, you can click on *Copy to Drive* button. You can alternatively click the *Share* button located at the top right corner and click on *Copy Link* under *Get Link* to get a link and copy this notebook to your Google Drive.
- For questions with descriptive answers, please replace the text in the cell which states "Enter your answer here!" with your answer. If you are using mathematical notation in your answers, please define the variables.
- For coding questions, you can add code where it says "enter code here" and execute the cell to print the output.
- **This assignment requires a GPU runtime.** In Colab, go to *Runtime -> Change runtime type* and select **T4 GPU**.
- To create the final pdf submission file, execute *Runtime->RunAll* from the menu to re-execute all the cells and then generate a PDF using *File->Print->Save as PDF*. Make sure that the generated PDF contains all the codes and printed outputs before submission.

**Submission Details**

- Due date: Wednesday April 15, 2026 at 11:59 PM (EDT).
- The final PDF file must be submitted to Gradescope.
- After copying this notebook to your Google Drive, please paste a link to it below. Use the same process given above to generate a link. ***You will not receive any credit if you don't paste the link!*** Make sure we can access the file.

***LINK:https://colab.research.google.com/drive/1cwHkBlkiXsb-uxKDkA-fPfFTL7HXAv1_?usp=sharing**

**Academic Honesty**

Please follow the guidelines under the *Collaboration and Help* section of the course website.

# Setup and Data Download

We use the ANTIQUE dataset for this assignment, consistent with previous assignments. We will also use pre-trained transformer models from the `sentence-transformers` library as the starting point for our bi-encoder.

**Please execute the cells below to install dependencies and download input files.**

```
!pip install -q sentence-transformers faiss-cpu datasets gdown
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.8/23.8 MB 93.8 MB/s eta
0:00:00
```

```
import os
import zipfile

# Download the ANTIQUE dataset files
!gdown 11K8r5a_Aj9S4Cpd8k3x76ccBTluNguip -O HW07.zip

with zipfile.ZipFile('HW07.zip', 'r') as zip_file:
    zip_file.extractall('./')

if os.path.exists('HW07.zip'):
    os.remove('HW07.zip')

os.chdir('HW07')

# Setting input files
passage_file = "antique-collection.tok.clean_kstem"
test_queries_file = "antique-test-queries.tok.clean_kstem"
train_queries_file = "antique-train-queries.tok.clean_kstem"
val_queries_file = "antique-val-queries.tok.clean_kstem"
train_baseline_features_file = "train_baseline_features_top10"
val_baseline_features_file = "val_baseline_features_top10"
test_baseline_features_file = "test_baseline_features_top10"
```

```
Downloading...
From (original): https://drive.google.com/uc?
id=11K8r5a_Aj9S4Cpd8k3x76ccBTluNguip
From (redirected): https://drive.google.com/uc?
id=11K8r5a_Aj9S4Cpd8k3x76ccBTluNguip&confirm=t&uuid=74d52cd2-6c24-
4dd2-ac5e-fcaf3abb5bf4
To: /content/HW07.zip
   0% 0.00/33.3M [00:00<?, ?B/s]  49% 16.3M/33.3M [00:00<00:00,
155MB/s]  96% 32.0M/33.3M [00:00<00:00, 150MB/s] 100% 33.3M/33.3M
[00:00<00:00, 152MB/s]
```

We also need the **original (unstemmed) versions** of the passages and queries for neural models, since transformer models use their own tokenizers. We provide these below.

**Note:** For this assignment, we will use the stemmed text as a reasonable proxy for the original text (the ANTIQUE dataset's original text). In a real setting, you would use the raw text. The models will still learn meaningful representations.

# 1: Data Loading and Preparation (15 points)

In this section, you will:

1. Load the collection, queries, and relevance judgments.
2. Create training triples (query, positive passage, negative passage) from the feature files.

The feature files contain `query_id passage_id relevance_score vsm_score bm25_score`. We will use `relevance_score` to determine positive and negative passages.

**Relevance levels:** The ANTIQUE dataset uses relevance scores from 1 to 4, where 4 is most relevant and 1 is least relevant. For training the bi-encoder:

- **Positive passages**: relevance score >= 3
- **Negative passages**: relevance score <= 2

If a query has no positive or no negative passage in its candidate set, skip that query.

```
'''
Load the passage collection.
Return:
    coll - dict mapping passage_id (str) to passage_text (str)
'''
def loadCollection(passage_file):
    # enter code here
    coll = {}
    with open(passage_file, 'r', encoding='utf-8') as file:
      for line in file:
        line = line.strip()
        if not line:
            continue

        passage_id, passage_text = line.strip().split('\t')
        coll[passage_id] = passage_text

    return coll

'''
Load a query file.
Return:
    queries - dict mapping query_id (str) to query_text (str)
'''
```

```
def loadQueryFile(filename):
    # enter code here
    queries = {}
    with open(filename, 'r', encoding='utf-8') as file:
      for line in file:
        line = line.strip()
        if not line:
            continue

        qid, query_text = line.strip().split('\t')
        queries[qid] = query_text

    return queries


coll = loadCollection(passage_file)
train_queries = loadQueryFile(train_queries_file)
val_queries = loadQueryFile(val_queries_file)
test_queries = loadQueryFile(test_queries_file)

print(f'Collection size: {len(coll)}')
print(f'Train queries: {len(train_queries)}')
print(f'Val queries: {len(val_queries)}')
print(f'Test queries: {len(test_queries)}')
```

```
Collection size: 403492
Train queries: 2226
Val queries: 200
Test queries: 200
```

```
'''
Parse a baseline features file and return relevance judgments.
Return:
    qrels - dict mapping query_id to a list of (passage_id,
relevance_score) tuples
'''
def loadFeatureFile(features_file):
    # enter code here
    qrels = {}
    with open(features_file, 'r', encoding='utf-8') as file:
      for line in file:
        line = line.strip()
        if not line:
            continue
        query_id, passage_id, relevance_score, vsm_score, bm25_score =
line.strip().split()
        if query_id not in qrels:
            qrels[query_id] = []
        qrels[query_id].append((passage_id, int(relevance_score)))
    return qrels
```

```
train_qrels = loadFeatureFile(train_baseline_features_file)
val_qrels = loadFeatureFile(val_baseline_features_file)
test_qrels = loadFeatureFile(test_baseline_features_file)

print(f'Train qrels: {len(train_qrels)} queries')
print(f'Val qrels: {len(val_qrels)} queries')
print(f'Test qrels: {len(test_qrels)} queries')
```

```
Train qrels: 2226 queries
Val qrels: 200 queries
Test qrels: 200 queries
```

```
'''
Create training triples from the qrels.
For each query:
  - Positive passages: those with relevance_score >= 3
  - Negative passages: those with relevance_score <= 2
  - Create all combinations of (query_text, positive_passage_text,
negative_passage_text)
  - Skip queries with no positives or no negatives.

Return:
    triples - list of tuples: (query_text, positive_passage_text,
negative_passage_text)
'''
def createTrainingTriples(queries, qrels, coll):
    triples = []

    for qid in qrels:
        # Skip if query text is missing
        if qid not in queries:
            continue

        positive_passages = []
        negative_passages = []

        for passage_id, relevance_score in qrels[qid]:
            # Skip passage IDs not found in collection
            if passage_id not in coll:
                continue

            if relevance_score >= 3:
                positive_passages.append(passage_id)
            elif relevance_score <= 2:
                negative_passages.append(passage_id)

        # Explicitly skip queries with no positives or no negatives
        if not positive_passages or not negative_passages:
            continue
```

```
        for positive_passage in positive_passages:
            for negative_passage in negative_passages:
                # Skip useless triples where positive and negative are
the same passage
                if positive_passage == negative_passage:
                    continue

                triples.append((
                    queries[qid],
                    coll[positive_passage],
                    coll[negative_passage]
                ))

    return triples


train_triples = createTrainingTriples(train_queries, train_qrels,
coll)
val_triples = createTrainingTriples(val_queries, val_qrels, coll)

print(f'Number of training triples: {len(train_triples)}')
print(f'Number of validation triples: {len(val_triples)}')
print(f'\nExample triple:')
print(f'  Query:    {train_triples[0][0][:80]}...')
print(f'  Positive: {train_triples[0][1][:80]}...')
print(f'  Negative: {train_triples[0][2][:80]}...')
```

```
Number of training triples: 16808
Number of validation triples: 1605

Example triple:
  Query:    how do i get them white...
  Positive: first seek professional help then work on your teeth
because no matter what you ...
  Negative: not me dude i do n t like pink inside my steak medium well
is how i order i do n...
```

# 2: Bi-Encoder Model (35 points)

In this section, you will implement and train a **bi-encoder** model for dense retrieval.

A bi-encoder encodes queries and passages **independently** into dense vectors, then uses dot product (or cosine similarity) to compute relevance scores:

$$score(q, p) = \text{Enc}_Q(q) \cdot \text{Enc}_P(p)$$

We will use `sentence-transformers` with the **`all-MiniLM-L6-v2`** model as our base encoder. This is a small but effective model suitable for the assignment.

## 2.1: Implement the Bi-Encoder Training (25 points)

You will fine-tune the bi-encoder using a **triplet margin loss**:

$$L = max¿$$

where $\text{sim}$ is cosine similarity, $p^{+¿¿}$ is a positive passage, $p^{-}$ is a negative passage, and margin is a hyperparameter (use **margin = 0.2**).

**Instructions:**

- Use `all-MiniLM-L6-v2` as the pre-trained model.
- Fine-tune for **2 epochs** with batch size **32** and learning rate **2e-5**.
- Use the `sentence_transformers` `SentenceTransformer`, `InputExample`, `DataLoader`, and `losses.TripletLoss`.
- Evaluate on the validation set after training.

```
import torch
from sentence_transformers import SentenceTransformer, InputExample,
losses
from torch.utils.data import DataLoader

'''
Create a list of InputExample objects for the sentence-transformers
library.
Each InputExample for TripletLoss takes texts=[anchor, positive,
negative].

Parameters:
    triples - list of (query_text, positive_text, negative_text)
tuples
Return:
    examples - list of InputExample
'''
def createInputExamples(triples):
    examples = []
    for query_text, positive_text, negative_text in triples:
        examples.append(InputExample(texts=[query_text, positive_text,
negative_text]))
    return examples


train_examples = createInputExamples(train_triples)
print(f'Number of training examples: {len(train_examples)}')
```

```
/tmp/ipykernel_656/1743069085.py:2: DeprecationWarning: Importing from
'sentence_transformers.losses' is deprecated and will be removed in a
future version. Please use
'sentence_transformers.sentence_transformer.losses' instead.
  from sentence_transformers import SentenceTransformer, InputExample,
losses
```

```
Number of training examples: 16808
```

```
'''
Initialize the bi-encoder model, create the dataloader and loss
function,
and fine-tune the model.

Steps:
1. Load the pre-trained model 'all-MiniLM-L6-v2'
2. Create a DataLoader from train_examples with batch_size=32,
shuffle=True
3. Use losses.TripletLoss with
distance_metric=losses.TripletDistanceMetric.COSINE
   and triplet_margin=0.2
4. Train for 2 epochs with warmup_steps=100, and
optimizer_params={'lr': 2e-5}
5. Save the model to 'biencoder_antique'

Return:
    model - the fine-tuned SentenceTransformer model
'''
BATCH_SIZE = 32

def trainBiEncoder(train_examples):
    # enter code here
    model = SentenceTransformer('all-MiniLM-L6-v2')
    train_dataloader = DataLoader(train_examples, shuffle=True,
batch_size=BATCH_SIZE)

    train_loss = losses.TripletLoss(model=model,
distance_metric=losses.TripletDistanceMetric.COSINE,
triplet_margin=0.2)
    model.fit(train_objectives=[(train_dataloader, train_loss)],
epochs=2, warmup_steps=100, optimizer_params={'lr': 2e-5})

    model.save('biencoder_antique')
    print('Model saved to biencoder_antique')

    return model


model = trainBiEncoder(train_examples)
print(f'Model embedding dimension:
{model.get_sentence_embedding_dimension()}')
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
```

```
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"9ac026b4cdb94cbbaa5e85f0d715a4a8","version_major":2,"vers
ion_minor":0}

{"model_id":"d15bdf051ee341b0970458cf50ef144c","version_major":2,"vers
ion_minor":0}

{"model_id":"f1a52a6de4ff4a9d88524e0f7f427f00","version_major":2,"vers
ion_minor":0}

{"model_id":"ddba29a29cf7468696c952098c541a19","version_major":2,"vers
ion_minor":0}

{"model_id":"b956c6be81d4413f9fa8e0143fc6fd72","version_major":2,"vers
ion_minor":0}

{"model_id":"bff85161af2b4e51af671606df8c998a","version_major":2,"vers
ion_minor":0}

{"model_id":"b20e0acd32a84cae8c95cdb30493c21c","version_major":2,"vers
ion_minor":0}
```


```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
```


```
{"model_id":"d005313e26084e889b585d9f3557ffae","version_major":2,"vers
ion_minor":0}

{"model_id":"454a9ac1861a4cffa2680db8b72fb205","version_major":2,"vers
ion_minor":0}

{"model_id":"435b7cc8a1d04f4da99e3eac2a152f48","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"53391aacb6554cafb15b547aa31435c4","version_major":2,"vers
ion_minor":0}

{"model_id":"a99a77f894244d40bb19099280046c5d","version_major":2,"vers
ion_minor":0}

{"model_id":"558742983a224eb682f13e0f3d742df9","version_major":2,"vers
ion_minor":0}

<IPython.core.display.HTML object>

{"model_id":"5bceb3fe1d80472eb0d810637dee4ba8","version_major":2,"vers
ion_minor":0}

Model saved to biencoder_antique
Model embedding dimension: 384

/tmp/ipykernel_656/1084284797.py:33: FutureWarning: The
`get_sentence_embedding_dimension` method has been renamed to
`get_embedding_dimension`.
  print(f'Model embedding dimension:
{model.get_sentence_embedding_dimension()}')
```

## 2.2: Encode and Evaluate (10 points)

Now encode all test queries and their candidate passages, compute cosine similarity scores, and evaluate using **NDCG@10**.

For each test query, rank its candidate passages (from `test_qrels`) by the bi-encoder similarity score and compute NDCG@10 using the relevance labels.

```
import numpy as np
from sklearn.metrics import ndcg_score

'''
Evaluate the bi-encoder on test queries using NDCG@10.

For each query in test_qrels:
1. Encode the query text using model.encode()
2. Encode all candidate passage texts for that query
3. Compute cosine similarity between the query and each candidate
passage
4. Use the relevance scores from test_qrels as ground truth
5. Compute NDCG@10

Parameters:
    model - SentenceTransformer model
    test_queries - dict of query_id -> query_text
    test_qrels - dict of query_id -> list of (passage_id,
relevance_score)
    coll - dict of passage_id -> passage_text
```

```
Return:
    mean_ndcg - float, mean NDCG@10 across all test queries
'''
def evaluateBiEncoder(model, test_queries, test_qrels, coll):
    ndcg_scores = []

    for qid in test_qrels:
        if qid not in test_queries:
            continue

        query_text = test_queries[qid]
        query_embedding = model.encode(query_text,
convert_to_numpy=True)

        relevance_scores = []
        sim_scores = []

        for passage_id, relevance_score in test_qrels[qid]:
            if passage_id not in coll:
                continue

            passage_text = coll[passage_id]
            passage_embedding = model.encode(passage_text,
convert_to_numpy=True)

            sim_score = np.dot(query_embedding, passage_embedding)

            relevance_scores.append(relevance_score)
            sim_scores.append(sim_score)

        if not relevance_scores:
            continue

        query_ndcg = ndcg_score([relevance_scores], [sim_scores],
k=10)
        ndcg_scores.append(query_ndcg)

    mean_ndcg = float(np.mean(ndcg_scores)) if ndcg_scores else 0.0
    return mean_ndcg


biencoder_ndcg = evaluateBiEncoder(model, test_queries, test_qrels,
coll)
print(f'Bi-Encoder NDCG@10 on test set: {biencoder_ndcg:.4f}')
```

```
Bi-Encoder NDCG@10 on test set: 0.8338
```

# 3: FAISS Indexing for Efficient Retrieval (30 points)

In a real retrieval system, we cannot afford to compare the query against every passage at query time. Instead, we build an **index** over all passage embeddings and use **approximate nearest neighbor (ANN)** search.

In this section, you will:

1. Encode the entire passage collection using the fine-tuned bi-encoder.
2. Build a FAISS index.
3. Retrieve passages for test queries using FAISS.
4. Evaluate retrieval quality.

## 3.1: Encode the Collection and Build FAISS Index (10 points)

Encode all passages in the collection and build two FAISS indices:

- **Flat (exact) index**: `faiss.IndexFlatIP` (inner product / dot product)
- **IVF (approximate) index**: `faiss.IndexIVFFlat` with `nlist=100` and `nprobe=10`

**Important:** Normalize all vectors to unit length before indexing so that inner product equals cosine similarity.

```
import faiss

'''
Encode the entire collection and return the embeddings and a mapping.

Steps:
1. Create an ordered list of (passage_id, passage_text) pairs.
2. Encode all passage texts using model.encode() with
normalize_embeddings=True,
   batch_size=256, and show_progress_bar=True.
3. Return the embeddings as a numpy float32 array and the ordered list
of passage_ids.

Parameters:
    model - SentenceTransformer model
    coll - dict of passage_id -> passage_text
Return:
    passage_embeddings - numpy array of shape (num_passages,
embedding_dim), dtype float32
    passage_ids - list of passage_id strings in the same order
'''
def encodeCollection(model, coll):
    ordered_passages = list(coll.items())
```

```
    passage_ids = [passage_id for passage_id, _ in ordered_passages]
    passage_texts = [passage_text for _, passage_text in
ordered_passages]

    passage_embeddings = model.encode(
        passage_texts,
        normalize_embeddings=True,
        batch_size=256,
        show_progress_bar=True,
        convert_to_numpy=True
    ).astype(np.float32)

    return passage_embeddings, passage_ids


passage_embeddings, passage_ids = encodeCollection(model, coll)
print(f'Encoded {len(passage_ids)} passages with shape
{passage_embeddings.shape}')
```

{"model_id":"2e1d69eba9a24657a792e7a238231bea","version_major":2,"vers
ion_minor":0}

```
Encoded 403492 passages with shape (403492, 384)
```

```
'''
Build a FAISS flat (exact) index and an IVF (approximate) index.

Steps:
1. Build a flat inner product index using faiss.IndexFlatIP(dim).
   Add all passage_embeddings to it.

2. Build an IVF index:
   a. Create a quantizer using faiss.IndexFlatIP(dim).
   b. Create the IVF index using faiss.IndexIVFFlat(quantizer, dim,
nlist, faiss.METRIC_INNER_PRODUCT)
      where nlist=100.
   c. Train the IVF index on the passage_embeddings.
   d. Add all passage_embeddings to the IVF index.
   e. Set nprobe=10 on the IVF index.

Parameters:
    passage_embeddings - numpy array of shape (N, D)
Return:
    flat_index - faiss.IndexFlatIP
    ivf_index - faiss.IndexIVFFlat
'''
def buildFaissIndices(passage_embeddings):
    passage_embeddings = passage_embeddings.astype(np.float32)

    dim = passage_embeddings.shape[1]
    flat_index = faiss.IndexFlatIP(dim)
```

```
    flat_index.add(passage_embeddings)

    nlist = 100
    quantizer = faiss.IndexFlatIP(dim)
    ivf_index = faiss.IndexIVFFlat(quantizer, dim, nlist,
faiss.METRIC_INNER_PRODUCT)
    ivf_index.train(passage_embeddings)
    ivf_index.add(passage_embeddings)
    ivf_index.nprobe = 10

    return flat_index, ivf_index


flat_index, ivf_index = buildFaissIndices(passage_embeddings)
print(f'Flat index size: {flat_index.ntotal}')
print(f'IVF index size: {ivf_index.ntotal}')
print(f'IVF index trained: {ivf_index.is_trained}')
```

```
Flat index size: 403492
IVF index size: 403492
IVF index trained: True
```

## 3.2: Retrieve and Evaluate with FAISS (20 points)

For each test query:

1. Encode the query with the bi-encoder (normalized).
2. Search both the flat and IVF indices for the top 10 passages.
3. Compute **NDCG@10** and **Recall@10** using the relevance judgments from `test_qrels`.

**Recall@10** is defined as the fraction of relevant passages (relevance >= 3) in the top-10 retrieved results out of all relevant passages for that query in `test_qrels`.

Also measure the **average query latency** (in milliseconds) for both flat and IVF indices.

```
import time
from sklearn.metrics import ndcg_score

'''
Evaluate FAISS retrieval on test queries.

For each query in test_queries:
1. Encode the query using model.encode() with
normalize_embeddings=True.
2. Search the given FAISS index for top-k=10 nearest neighbors.
3. Map the returned indices back to passage_ids using the passage_ids
list.
4. Look up relevance scores from test_qrels. Passages not in
test_qrels get score 0.
5. Compute NDCG@10 (using sklearn.metrics.ndcg_score) and Recall@10.
```

```
6. Track total search time (exclude encoding time) to compute average
latency.

Parameters:
    model - SentenceTransformer model
    index - a FAISS index
    test_queries - dict of query_id -> query_text
    test_qrels - dict of query_id -> list of (passage_id,
relevance_score)
    passage_ids - list of passage_id strings
    k - number of results to retrieve (default 10)

Return:
    mean_ndcg - float
    mean_recall - float
    avg_latency_ms - float, average search latency in milliseconds per
query
'''
def evaluateFaissRetrieval(model, index, test_queries, test_qrels,
passage_ids, k=10):
    ndcg_scores = []
    recall_scores = []
    total_search_time = 0.0
    num_queries = 0

    for qid in test_queries:
        if qid not in test_qrels:
            continue

        query_text = test_queries[qid]
        query_embedding = model.encode(
            query_text,
            normalize_embeddings=True,
            convert_to_numpy=True
        ).astype(np.float32).reshape(1, -1)

        start_time = time.time()
        scores, indices = index.search(query_embedding, k)
        end_time = time.time()

        total_search_time += (end_time - start_time) * 1000

        rel_dict = {pid: rel for pid, rel in test_qrels[qid]}

        retrieved_rels = []
        retrieved_scores = []
        retrieved_relevant_count = 0

        for score, idx in zip(scores[0], indices[0]):
            if idx == -1:
```

```
                continue

            retrieved_pid = passage_ids[idx]
            rel = rel_dict.get(retrieved_pid, 0)

            retrieved_rels.append(rel)
            retrieved_scores.append(score)

            if rel >= 3:
                retrieved_relevant_count += 1

        if not retrieved_rels:
            continue

        total_relevant = sum(1 for _, rel in test_qrels[qid] if rel >=
3)
        if total_relevant == 0:
            continue

        query_ndcg = ndcg_score([retrieved_rels], [retrieved_scores],
k=k)
        query_recall = retrieved_relevant_count / total_relevant

        ndcg_scores.append(query_ndcg)
        recall_scores.append(query_recall)
        num_queries += 1

    mean_ndcg = float(np.mean(ndcg_scores)) if ndcg_scores else 0.0
    mean_recall = float(np.mean(recall_scores)) if recall_scores else
0.0
    avg_latency_ms = total_search_time / num_queries if num_queries >
0 else 0.0

    return mean_ndcg, mean_recall, avg_latency_ms


print('=== Flat (Exact) Index ===')
flat_ndcg, flat_recall, flat_latency = evaluateFaissRetrieval(
    model, flat_index, test_queries, test_qrels, passage_ids
)
print(f'  NDCG@10:           {flat_ndcg:.4f}')
print(f'  Recall@10:         {flat_recall:.4f}')
print(f'  Avg latency (ms):  {flat_latency:.2f}')

print('\n=== IVF (Approximate) Index ===')
ivf_ndcg, ivf_recall, ivf_latency = evaluateFaissRetrieval(
    model, ivf_index, test_queries, test_qrels, passage_ids
)
print(f'  NDCG@10:           {ivf_ndcg:.4f}')
```

```
print(f'  Recall@10:          {ivf_recall:.4f}')
print(f'  Avg latency (ms):   {ivf_latency:.2f}')
```

```
=== Flat (Exact) Index ===
  NDCG@10:           0.5385
  Recall@10:         0.4414
  Avg latency (ms):  84.68

=== IVF (Approximate) Index ===
  NDCG@10:           0.5063
  Recall@10:         0.3769
  Avg latency (ms):  9.31
```

# 4: Comparison with BM25 Baseline (20 points)

Now let's compare the bi-encoder dense retrieval approach with the BM25 baseline.

## 4.1: BM25 Re-ranking Evaluation (10 points)

Using the BM25 scores from the test feature file, compute NDCG@10 for BM25 re-ranking on the test queries. Then print a comparison table.

```
from collections import defaultdict
'''
Compute BM25 NDCG@10 on test queries using the scores in the feature
file.

Read the test_baseline_features_file. For each query, rank candidates
by
their bm25_score (the 5th column). Compute NDCG@10 using
relevance_score
(the 3rd column) as ground truth.

Return:
    mean_ndcg - float, mean NDCG@10
'''
def evaluateBM25(features_file):
    # enter code here
    qid_to_rels = defaultdict(list)
    qid_to_bm25 = defaultdict(list)

    with open(features_file, 'r', encoding='utf-8') as file:
        for line in file:
            line = line.strip()
            if not line:
                continue

            query_id, passage_id, relevance_score, vsm_score,
```

```
bm25_score = line.split()

            qid_to_rels[query_id].append(int(relevance_score))
            qid_to_bm25[query_id].append(float(bm25_score))

    ndcg_scores = []

    for qid in qid_to_rels:
        y_true = [qid_to_rels[qid]]
        y_score = [qid_to_bm25[qid]]

        query_ndcg = ndcg_score(y_true, y_score, k=10)
        ndcg_scores.append(query_ndcg)

    mean_ndcg = float(np.mean(ndcg_scores)) if ndcg_scores else 0.0

    return mean_ndcg


bm25_ndcg = evaluateBM25(test_baseline_features_file)
print(f'BM25 NDCG@10: {bm25_ndcg:.4f}')
```

```
BM25 NDCG@10: 0.8525
```

```
# Print comparison table
print(f'{"Method":<30} {"NDCG@10":<12}')
print('-' * 42)
print(f'{"BM25 (re-ranking)":<30} {bm25_ndcg:<12.4f}')
print(f'{"Bi-Encoder (re-ranking)":<30} {biencoder_ndcg:<12.4f}')
print(f'{"Bi-Encoder + Flat FAISS":<30} {flat_ndcg:<12.4f}')
print(f'{"Bi-Encoder + IVF FAISS":<30} {ivf_ndcg:<12.4f}')
```

```
Method                         NDCG@10
------------------------------------------
BM25 (re-ranking)              0.8525
Bi-Encoder (re-ranking)        0.8338
Bi-Encoder + Flat FAISS        0.5385
Bi-Encoder + IVF FAISS         0.5063
```

## 4.2: Analysis (10 points)

Based on the results above, answer the following:

1. How does the bi-encoder's NDCG@10 in re-ranking mode (Section 2.2) compare to BM25? Provide a possible explanation for the difference.

2. How does the NDCG@10 from FAISS full-collection retrieval (Section 3.2) compare to the re-ranking NDCG@10 (Section 2.2)? Why might they differ?

3. Answer: From the result, we noticed that the bi-encoder achieves slightly lower NDCG@10 (0.8338) compares to BM25 (0.8525). One possible explanation is that

BM25 is a strong lexical matching baseline that will perform really well on datasets like ANTIQUE, where queries often share exact terms with relevant passages. In contrast, the bi-encoder relies on semantic similarity and was only fine-tuned for a small number of epochs (2 epochs) with limited training data, which may not be sufficient to outperform BM25. Moreover, since the evaluation is done in a re-ranking setting, BM25 already provides a strong candidate set, leaving limited room for improvement.

4. Answer: The NDCG@10 from FAISS retreival (0.5385 for Flat and 0.5063 for IVF) is significantly lower than the re-ranking performance (0.8338). This difference arises because re-ranking operates on a high-quality candidate set (typically retreived by BM25), where as FAISS retrieval must search over the entire collection. This means that errors in retrieving relevant passages cannot be corrected later, leading to lower performance. Additionally the bi-encoder model may no be strong enough to perform effective full-collection retrieval. The IVF index further reduces accuracy compared to the flat index due to its approximate nature, though it achieves much lower latency, illustrating the trade-off between efficiency and retrieval quality